
UNIT 2: DATA CLEANING AND PREPARATION

Structure	Page No.
2.0 Introduction	27
2.1 Objectives	28
2.2 Fundamentals of Data Cleaning and Preparation	28
2.2.1 What is Data Cleaning?	28
2.2.2 Data Cleaning vs. Data Transformation	29
2.3 Handling Missing Data (Imputation and Deletion)	30
2.3.1 Identifying and Understanding Missing Data	30
2.3.2 Filtering Out Missing Data	35
2.3.3 Filling in Missing Data	42
2.4 Data Transformation	46
2.4.1 Finding and Removing Duplicate Records	46
2.4.2 Transforming Data: apply versus transform versus map	51
2.4.3 Replacing Values and Renaming Columns or Row Indexes	54
2.5 Summary	58
2.6 Solutions/Answers	59
2.7 References and Further Readings	62

2.0 INTRODUCTION

Welcome to the world of Data Science! We are beginning our journey with perhaps the single most critical and time-consuming step in any data project: **Data Cleaning and Preparation**.

Consider data as the fuel that drives our data analysis and machine learning. You must carefully prepare your data before using it to produce trustworthy insights or create strong models, just as an engine needs clean gasoline to run effectively. A crucial and frequently the most time-consuming stage of the data science workflow, **data cleaning and preparation** is more than just a preliminary step. You run the danger of making serious mistakes and drawing incorrect conclusions if you skip this important step.

Rarely are raw datasets flawless by design. You will find that data originates from various sources, such as customer relationship management systems, sales reports, or manual entry, all of which introduce opportunities for errors. When working with data, you could run into frequent quality problems including inconsistent formatting, typos, damaged data points, and duplicate or missing entries. And when you combine data from multiple sources, these inconsistencies are often compounded. By addressing these problems methodically, you can make sure that your data is accurate, comprehensive, consistent, and suitable for analysis or decision-making.

When working with data, you need to be aware of a basic risk that is encapsulated in the "garbage in, garbage out" notion. The accuracy of any analysis or the performance of a machine learning model is directly dependent on the quality of the data you use for training and testing. When you base your decisions on dirty data, data containing duplicates, typos, or inconsistencies, you may experience strategic errors, resource waste, and lost commercial prospects. For machine learning models, if you train them

with poor quality data, you will probably observe inaccurate predictions and poor generalization to fresh, untested data. Consequently, let us emphasise that data cleaning is an essential process for guaranteeing that the results of any data-driven project are both meaningful and actionable.

2.1 OBJECTIVES

After you study this unit and complete the exercises, you will be able to:

- Explain why data cleaning and preparation matter so much in the data science lifecycle and how they contribute to successful outcomes.
- Identify the different types of missing data you may encounter and understand what implications each type carries for your analysis.
- Use a variety of strategies to deal with the missing data you discover, including as imputation techniques and filtering strategies that are most appropriate for your circumstances.
- Use pandas functions to help you identify and remove duplicate entries from your dataset efficiently.
- Transform and manipulate your data using custom functions, mapping strategies, and value replacement techniques to prepare it for analysis.
- Rename your row and column indices in an efficient manner to guarantee consistency and clarity across your collection.
- Perform quality checks on your cleaned dataset and validate it thoroughly to confirm it meets the standards required for reliable analysis.

2.2 FUNDAMENTALS OF DATA CLEANING AND PREPARATION

Data cleaning and preparation is the essential first step in the data science lifecycle. At this stage, we focus on getting your raw data ready for analysis by identifying and addressing quality issues such as errors, missing values, duplicates, and inconsistencies. In this section, we will first clarify what we mean by data cleaning (2.3.1), and then we will distinguish it from the related but separate step of data transformation (2.3.2). As you read, you may find it helpful to think about how you would apply each idea to a dataset you have worked with.

2.2.1 What is Data Cleaning?

When we talk about data cleaning, we are referring to the process of detecting and correcting or removing corrupt, inaccurate, duplicate, or irrelevant records from a dataset. The purpose of this process is to improve data quality so that your dataset is truly ready for analysis. Let us look at the main activities you will typically perform when you clean data:

- **Identifying and removing errors and inconsistencies:** Here, you focus on fixing structural issues such as unusual naming conventions, typos, or inconsistent capitalisation. For example, you may decide to standardise values like “N/A” and “Not Applicable” into a single category so that your analysis is consistent.

- **Filling in missing values:** In this step, you deal with gaps in the data where information is missing, perhaps because of incomplete data collection or system problems. Depending on the situation, you can use a variety of methods to deal with these missing numbers.
- **Handling outliers:** You look at data points that deviate significantly from the norm. Together, we need to decide whether there are significant anomalies that should be retained and carefully evaluated, or mistakes that should be fixed or eliminated.
- **Removing duplicates:** Finally, you look for repeated records that represent the same observation. Removing these duplicates helps you avoid inflated counts and biased results.

You are progressively transforming your dataset from a "dirty" state to one that is reliable, accurate, and useful as you complete these tasks.

2.2.2 Data Cleaning vs. Data Transformation

If you are new to data science, you may be tempted to treat data cleaning and data transformation as the same thing. However, it is crucial that we maintain their conceptual separation, because they occur in sequence and serve different purposes in the data preparation workflow.

The goal of **data cleaning** is to correct errors in the raw data. In this phase, you work on enforcing integrity and consistency by dealing with typos, missing values, duplicates, and other quality issues. The goal is to ensure that the data you keep is trustworthy.

Data transformation, by contrast, starts after the data has been cleaned. At this stage, we assume the data is reliable, and we transform it into a form that is more suitable for a particular analysis or machine learning model. Here, you may:

- **Standardise variables**, for example, by converting all measurements into the same unit.
- **Aggregate data**, such as summarising daily records into weekly or monthly totals.
- **Create derived variables**, like computing a "total_revenue" column by multiplying "price" by "quantity".

If we think about the overall workflow, the steps usually follow a logical order. First, data is collected. Next, you profile the data to identify potential issues. Then, you carry out the cleaning process to correct those issues. Only after the data is clean do you move on to structuring and transforming it for final analysis or modeling. This two-stage approach helps ensure that any transformation you apply is built on reliable data, rather than amplifying errors from a messy dataset.

Check Your Progress 1

1. Why is data cleaning considered the most time-consuming yet crucial step in data science projects? (3-5 lines)

.....

.....

.....

2. We have data where India, india, and IN are all used to represent the same country. Which primary data cleaning activity would address this issue?

.....
.....
.....

3. State whether the following statement is True or False:

Data Transformation is performed before Data Cleaning to ensure the data is standardised for error detection.

.....
.....

4. If a dataset includes a column for purchase_price and discount_amount, and you create a new column called final_price using these two, what type of activity is this?

.....
.....

2.3 HANDLING MISSING DATA (IMPUTATION AND DELETION)

Missing data refers to the absence of a value for a variable in a dataset, commonly represented as NaN (Not a Number) or None in Pandas, which must be addressed before analytical algorithms can be applied. This critical section is divided into three key steps for managing these gaps:

1. Identifying the missing values and classifying their underlying nature (2.4.1),
2. Filtering out incomplete rows or columns using deletion methods (2.4.2), and
3. Employing imputation techniques, such as statistical measures, to fill in the absent values (2.4.3).

2.3.1 Identifying and Understanding Missing Data

The approach we take to handle missing data must be guided by why the data is missing. Understanding this mechanism is vital, as ignoring it can introduce significant bias into our analysis.

Missing Data Samples and Reasons

Let us discuss the missing data issues with the help of an example.

Emergency Department Triage Log

Let us look at how emergency care settings frequently have missing data. Here, you may observe that staff members occasionally omit non-essential fields during data entry due to the intense time constraints and large patient load. As a result, missing data regularly occurs—not necessarily due to mistakes, but rather as a result of clinical teams concentrating on gathering only the most crucial information when time is of the essence.

patient_arrival_time	triage_level	initial_complaint	allergies_recorded	weight_kg	systolic_bp
2024-09-28 8:15:00	1	Chest Pain	PENICILLIN	85.5	130
2024-09-28 8:20:00	4	Ankle Sprain		92.1	145
2024-09-28 8:25:00	2	Severe Headache	NONE		160
2024-09-28 8:30:00	1	Trauma	SHELLFISH		
2024-09-28 8:35:00	3	Fever	NONE	65	
2024-09-28 8:40:00	5	Sore Throat	NONE	72.3	125

Remote Telehealth Monitoring

When you collect data remotely, you may find that device problems, internet issues, or even the way a patient follows instructions can all affect the completeness of your records. Because of these factors, we sometimes see entire sections of data missing. This happens not only due to technical glitches but also because patients might forget or choose not to provide certain information. Let us keep these challenges in mind as we try to work with and interpret remotely gathered data.

telehealth_id	date	systolic_bp	spo2_percentage	time_of_reading
T401	2024-09-28	125	97	8:15:00
T402	2024-09-28		98	
T403	2024-09-28	140		8:00:00
T401	2024-09-29			
T402	2024-09-29	135	97	8:30:00
T403	2024-09-29	142	96	8:05:00

Hospital ICU Monitoring Data

This data comes from continuous monitoring equipment in a high-intensity environment like an ICU, where hardware failures are common.

	admission_id	minute_timestamp	heart_rate	systolic_bp	respiratory_rate
0	I201	2025-07-06 10:00	85.0	120.0	18.0
1	I201	2025-07-06 10:03	87.0	118.0	18.0
2	I202	2025-07-06 10:01	75.0	NaN	16.0
3	I201	2025-07-06 10:07	NaN	122.0	17.0
4	I202	2025-07-06 10:05	78.0	125.0	NaN
5	I202	2025-07-06 10:09	79.0	124.0	16.0

The first step in handling missing data is to identify where the gaps exist. The pandas library provides intuitive functions for this:

Let us start by identifying missing values using pandas:

```
# Importing necessary libraries
import pandas as pd
import numpy as np
```

At first we import the important Python libraries,

- **pandas (pd)** — Used for data manipulation and analysis.
- **numpy (np)** — Provides numerical functions, including handling missing data with **np.nan**.

```
# Create a sample dataset with missing values
icu_data = {
    'admission_id': ['I201', 'I201', 'I202', 'I201', 'I202',
                    'I202'],
    'minute_timestamp': [
        "2025-07-06 10:00",
        "2025-07-06 10:03",
        "2025-07-06 10:01",
        "2025-07-06 10:07",
        "2025-07-06 10:05",
        "2025-07-06 10:09"
    ],
    'heart_rate': [85, 87, 75, np.nan, 78, 79], # Missing due
    # to detached ECG lead (I201, T+3min)
    'systolic_bp': [120, 118, np.nan, 122, 125, 124], #
    # Missing due to patient movement/skipped cuff cycle (I202,
    # T+2min)
    'respiratory_rate': [18, 18, 16, 17, np.nan, 16] #
    # Missing due to temporary ventilator disconnect (I202,
    # T+4min)
}
df = pd.DataFrame(icu_data)
```

Next we will create a panda Dataframe from a python dictionary using the pandas library. **np.nan** - indicates missing readings due to equipment failure or incomplete patient record entry.

```
print("\n--- Hospital ICU Monitoring Data (Equipment
Failure) ---")
print(df)
```

Finally, we have displayed the contents of the ICU DataFrame, to check if the data has been correctly converted.

Output of Program:

	admission_id	minute_timestamp	heart_rate	systolic_bp	respiratory_rate
0	I201	2025-07-06 10:00	85.0	120.0	18.0
1	I201	2025-07-06 10:03	87.0	118.0	18.0
2	I202	2025-07-06 10:01	75.0	NaN	16.0
3	I201	2025-07-06 10:07	NaN	122.0	17.0
4	I202	2025-07-06 10:05	78.0	125.0	NaN
5	I202	2025-07-06 10:09	79.0	124.0	16.0

The following segment detects the missing value from the program above.

```
# Detect Missing Values
# Identifying missing values using.isna() or.isnull()
# Both functions work the same way and return a Boolean
DataFrame
```

```
missing_values_df = df.isna()
print("\nDataFrame showing missing values (True for NaN):")
print(missing_values_df)
```

The panda function `isna()` is used to find out the missing values in the Dataframe. The function creates a Dataframe containing only Boolean data.

Output:

```
DataFrame showing missing values (True for NaN):
  admission_id  minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0         False                False        False        False            False
1         False                False        False        False            False
2         False                False        False         True            False
3         False                False         True        False            False
4         False                False        False        False            True
5         False                False        False        False            False
```

```
# A more practical approach is to count missing values per
column
missing_counts = df.isnull().sum()
print("\nTotal missing values per column:")
print(missing_counts)
```

Output:

```
Total missing values per column:
admission_id      0
minute_timestamp  0
heart_rate        1
systolic_bp       1
respiratory_rate  1
dtype: int64
```

```
# We can also calculate the percentage of missing values
total_rows = len(df)
percentage_missing = (missing_counts / total_rows) * 100
print("\nPercentage of missing values per column:")
print(percentage_missing)
```

Output:

```
Percentage of missing values per column:
admission_id      0.000000
minute_timestamp  0.000000
heart_rate        16.666667
systolic_bp       16.666667
respiratory_rate  16.666667
dtype: float64
```

When we deal with missing data, our first step should always be to ask: "Why is it missing?" As we have seen, the reason behind the gaps has a direct impact on whether your final conclusions are valid or unintentionally biased.

Let us try to break down the three main categories statisticians use to describe these missing values. You can select the most effective method for cleaning your dataset by being aware of these.

The Three Types of Missing Data

- **Missing Completely at Random (MCAR):** Think of this as a total fluke. The missing values are dispersed at random, either due to a momentary equipment malfunction or a data entry error. Because it's random, your remaining data is still a fair representation of the whole group.
- **Missing at Random (MAR):** This one is a bit tricky. The "missingness" isn't random, but it can be explained by other information we *did* collect. For example, if men are less likely to finish a survey than women, the gap is related to gender (which we know), rather than the survey answers themselves.
- **Missing Not at Random (MNAR):** This is the most challenging type. Here, the data is missing because of the value itself. For instance, people with very high or very low incomes might intentionally skip a question about their salary. This can lead to a heavily biased dataset that doesn't reflect the real world.

At a Glance: Comparing the Categories

You may find this table helpful for quickly identifying which situation you are dealing with:

Type of Missingness	Simple Definition	Common Cause	Impact on Your Analysis
MCAR	Unrelated to any data.	Random glitches or typos.	Low risk. Your remaining data stays unbiased.
MAR	Related to other known data points.	Specific groups (like a certain age) skipping questions.	Moderate risk. You may need special techniques to avoid bias.
MNAR	Related to the missing value itself.	Sensitive topics (income, health) being avoided.	High risk. The sample becomes unrepresentative and can lead to flawed conclusions.

By identifying which category your "dirty" data falls into, you can decide whether you should simply delete the missing rows or use a more advanced method to fill them in.

2.3.2 Filtering Out Missing Data

Once we have identified the missing values, we face a decision: do we remove the incomplete records or attempt to fill them? Filtering, or dropping data, is often the safest route when the amount of missingness is small or when the missing data occurs in a critical variable that cannot be reliably estimated.

The `dropna()` method is your primary tool for this. However, you must use it with caution. By default, `df.dropna()` will remove any row that contains even a single NaN value. This aggressive approach might lead to the loss of valuable information. Imagine

a row where only the 'Age' is missing, but the 'Python_Score' and 'Student_ID' are valid. By dropping the entire row, we lose the score data.

To prevent this, Pandas offers several parameters:

`DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False, ignore_index=False)`

Parameter	Type	Description	Default
axis	int or str	Specifies row-wise (0 or 'index') or column-wise (1 or 'columns') removal.	0
how	str	Condition for dropping: 'any' (default) or 'all'.	'any'
thresh	int	Minimum number of non-missing values required to retain the entity.	None
subset	list	List of labels along the other axis to consider for missingness checks.	None
inplace	bool	Whether to modify the original object or return a copy.	False
ignore_index	bool	Whether to reset the resulting index to sequential integers.	False

To illustrate these concepts, a clinical dataset scenario is established. In healthcare informatics, vital signs such as heart rate, blood pressure, and respiratory rate are recorded at frequent intervals. Sensor malfunctions or human errors often result in incomplete records. The following Python script initializes a sample DataFrame for use in Jupyter Notebook environments to demonstrate filtering techniques.

```
import pandas as pd
import numpy as np
# Initializing a clinical dataset with missing values
# Rows represent observations at specific minute intervals
data = {
    'admission_id': ['I201', 'I201', 'I202', 'I201', 'I202',
                    'I202'],
    'minute_timestamp': ['10:00', '10:03', '10:01', '10:07',
                        '10:05', '10:09'],
    'heart_rate': [85.0, 87.0, 75.0, np.nan, 78.0, 79.0],
    'systolic_bp': [120.0, 118.0, np.nan, 122.0, 125.0,
                    124.0],
    'respiratory_rate': [18.0, 18.0, 16.0, 17.0, np.nan,
                        16.0]
}
df = pd.DataFrame(data)
print("Original Clinical DataFrame:")
print(df)
```

The resulting DataFrame demonstrates varied patterns of missingness across both rows and columns. This provides the necessary foundation for exploring specific filtering arguments.

axis: What to Drop (Rows or Columns)

The `axis` parameter is the primary switch determining whether rows or columns are being targeted for removal. The default behavior of `dropna()` is to focus on rows. This is mathematically represented as an operation across the index. In most data science applications, rows represent individual samples or observations. Dropping a row implies that the entire observation is being discarded because it is incomplete.

```
# Dropping all rows that contain at least one missing value
df_rows_dropped = df.dropna()
print("DataFrame after dropping rows with any NaN
(axis=0):")
print(df_rows_dropped)
```

Output:

```
DataFrame after dropping rows with any NaN (axis=0):
  admission_id minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0          I201          10:00         85.0         120.0             18.0
1          I201          10:03         87.0         118.0             18.0
5          I202          10:09         79.0         124.0             16.0
```

In the clinical example, rows 2, 3, and 4 are removed because each contains at least one NaN in vital sign columns. The resulting DataFrame only retains rows 0, 1, and 5, where data is completely present for all features.

Column-wise Removal (axis=1)

By setting `axis=1`, the function targets columns. This is a more drastic measure typically reserved for features (variables) that are so sparsely populated they provide no statistical value.

```
# Dropping columns that contain at least one missing value
df_cols_dropped = df.dropna(axis=1)
print("DataFrame after dropping columns with any NaN
(axis=1):")
print(df_cols_dropped)
```

Output:

```
DataFrame after dropping columns with any NaN (axis=1):
  admission_id minute_timestamp
0          I201          10:00
1          I201          10:03
2          I202          10:01
3          I201          10:07
4          I202          10:05
5          I202          10:09
```

In the sample dataset, the columns `heart_rate`, `systolic_bp`, and `respiratory_rate` each contain at least one missing value. Consequently, they are entirely removed, leaving only `admission_id` and `minute_timestamp`. This highlights the risk of information loss, as the primary variables for analysis (vital signs) have been eliminated.

how: The Condition for Dropping

The `how` parameter specifies the rule for determining when a row or column should be dropped. It essentially sets the sensitivity of the filter.

Sensitivity Levels: 'any' vs 'all'

The parameter accepts two specific string values:

1. **'any' (Default):** The row or column is removed if at least one null value is present. This is the strictest setting and is used when any missingness renders the entire record invalid.
2. **'all':** The row or column is removed only if every single value in that axis is null. This is the most lenient setting and is used to remove purely empty placeholders without losing records that contain partial information.

To demonstrate `how='all'`, an entirely empty row must be added to the clinical dataset:

```
# Creating an extension with a row containing all NaNs
df_all_nan = pd.DataFrame([[np.nan, np.nan, np.nan, np.nan,
np.nan]], columns=df.columns)
df_ext = pd.concat([df, df_all_nan], ignore_index=True)

# Dropping rows only if ALL values are missing
df_all_rows_dropped = df_ext.dropna(how='all')
print("DataFrame after dropping rows where ALL values are
NaN:")
print(df_all_rows_dropped)
```

Output:

```
DataFrame after dropping rows where ALL values are NaN:
  admission_id minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0          I201         10:00         85.0         120.0             18.0
1          I201         10:03         87.0         118.0             18.0
2          I202         10:01         75.0           NaN             16.0
3          I201         10:07          NaN         122.0             17.0
4          I202         10:05         78.0         125.0             NaN
5          I202         10:09         79.0         124.0             16.0
```

In this operation, only the newly added row (index 6) is removed. Rows 2, 3, and 4, which were previously dropped by the default `how='any'`, are retained here because they contain at least some non-null values.

Implications for MultiIndex and Series

The `how` logic also applies to more complex pandas structures. In a Series, `how` is theoretically present for compatibility but functionally unnecessary because a Series only has one column of values; thus, 'any' and 'all' yield the same result. In a MultiIndex, `how` determines if an index entry is dropped when any level is NaN or only when all levels are NaN.

Structure	Logic of <code>how='any'</code>	Logic of <code>how='all'</code>
DataFrame (Rows)	Drop if any column in the row is NaN.	Drop only if all columns in the row are NaN.
DataFrame (Cols)	Drop if any row in the column is NaN.	Drop only if all rows in the column are NaN.
Index	Drop entry if any level is NaN.	Drop entry only if all levels are NaN.

subset: Targeted Filtering

A significant advancement in data cleaning involves targeting specific columns (when axis=0) or rows (when axis=1) to check for missingness. This prevents the unnecessary removal of rows that have missing values in non-essential columns.

Defining Mandatory Variables

In the clinical study, admission_id and minute_timestamp are considered mandatory metadata. However, the vital signs might have different levels of priority. If a research question focuses exclusively on the relationship between blood pressure and respiratory rate, missingness in the heart_rate column might be acceptable.

```
# Dropping rows only if 'systolic_bp' or 'respiratory_rate'
# are missing
df_subset_dropped = df.dropna(subset=['systolic_bp',
'systolic_bp',
'respiratory_rate'])
print("DataFrame after subset filtering on BP and Resp
Rate:")
print(df_subset_dropped)
```

Output:

DataFrame after subset filtering on BP and Resp Rate:						
	admission_id	minute_timestamp	heart_rate	systolic_bp	respiratory_rate	
0	I201	10:00	85.0	120.0	18.0	
1	I201	10:03	87.0	118.0	18.0	
3	I201	10:07	NaN	122.0	17.0	
5	I202	10:09	79.0	124.0	16.0	

The output of this program shows that row 3 is retained. Although it has a NaN in the heart_rate column, it contains valid values for both systolic_bp and respiratory_rate. Row 2 is removed because it has a NaN in systolic_bp, and row 4 is removed because it has a NaN in respiratory_rate.

Column-wise Subset Filtering

The subset parameter can also be utilized with axis=1. In this context, it allows the analyst to drop a column based on the presence of nulls in specific rows. For example, if the first five rows represent a "gold standard" calibration period, any column missing data during that period could be deemed unreliable and removed.

thresh: Quantitative Density

The thresh argument provides a quantitative threshold for retaining a row or column. It requires an integer value representing the minimum number of non-missing (valid) values that must be present.

Mathematical Thresholding Logic

For a DataFrame with N columns, setting thresh= K ensures that only rows with at least K non-null values are kept. This is particularly useful for filtering out observations that are too sparse to be statistically significant.

$$Keeprowif \sum_{i=1}^N [x_i is not null] \geq thresh$$

```
# Keeping rows that have at least 4 non-null values
df_thresh = df.dropna(thresh=4)
print("DataFrame after applying thresh=4 (Requires 4 non-
null values):")
print(df_thresh)
```

Output:

```
DataFrame after applying thresh=4 (Requires 4 non-null values):
  admission_id minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0          I201         10:00         85.0         120.0             18.0
1          I201         10:03         87.0         118.0             18.0
2          I202         10:01         75.0           NaN             16.0
3          I201         10:07           NaN         122.0             17.0
4          I202         10:05         78.0         125.0             NaN
5          I202         10:09         79.0         124.0             16.0
```

In the clinical DataFrame, each row has 5 potential values. Setting `thresh=4` effectively allows for one missing value per row. If a row has two or more missing values, it is discarded. Row 2 (missing `systolic_bp`) and row 3 (missing `heart_rate`) would both be kept under `thresh=4` because they each have 4 non-null values. However, if a row was missing both `heart_rate` and `systolic_bp`, it would be dropped.

Mutual Exclusivity: Thresh vs How

A critical constraint in pandas is that the `thresh` and `how` parameters are mutually exclusive. The analyst must choose between a qualitative rule ('any' or 'all') and a quantitative threshold. Attempting to pass both to the `dropna()` function will result in a logical error, as the system cannot prioritize between a general rule and a specific numerical count.

inplace: Non-destructive

By default, `dropna()` returns a new DataFrame object. This is a "non-destructive" operation that allows the original data to be preserved for comparison or alternative cleaning methods. However, when working with Big Data architectures, memory constraints are a significant concern.

Setting `inplace=True` modifies the original DataFrame directly and returns `None`. This avoids the creation of an intermediate copy in memory, which is essential for datasets with millions of rows.

```
# Modifying the original DataFrame directly to save memory
df_copy = df.copy()
df_copy.dropna(inplace=True)
print("Original DataFrame copy after inplace=True:")
print(df_copy)
```

Output:

```
Original DataFrame copy after inplace=True:
  admission_id minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0          I201         10:00         85.0         120.0             18.0
1          I201         10:03         87.0         118.0             18.0
5          I202         10:09         79.0         124.0             16.0
```

ignore_index: Relabeling axis

Removing rows creates gaps in the index of the DataFrame. For example, if row 2 is dropped, the index sequence jumps from 1 to 3. This can cause issues with future operations like positional slicing (`iloc`) or merging.

The `ignore_index` parameter, introduced in version 2.0, allows for the automatic relabeling of the resulting axis from 0 to `n-1`.

```
# Dropping rows and resetting the index automatically
df_ignored = df.dropna(ignore_index=True)
print("DataFrame with index reset through
ignore_index=True:")
print(df_ignored)
```

Output:

```
DataFrame with index reset through ignore_index=True:
  admission_id minute_timestamp  heart_rate  systolic_bp  respiratory_rate
0          I201           10:00         85.0         120.0             18.0
1          I201           10:03         87.0         118.0             18.0
2          I202           10:09         79.0         124.0             16.0
```

Prior to this feature, analysts had to use the separate `reset_index(drop=True)` method. The inclusion of this argument within `dropna()` streamlines the data cleaning workflow.

Comparative Analysis of Filtering Strategies

The selection of a filtering strategy depends on the sparsity of the data and the specific requirements of the analysis. The following table compares the implications of different parameter combinations.

Strategy	Sensitivity	Retention Level	Best Use Case
Default (how='any')	High	Low	When complete data is mandatory for every feature.
how='all'	Low	High	Removing purely empty rows added during data concatenation.
thresh=k	Medium	Medium	Maintaining a minimum amount of evidence per observation.
subset=[List]	Targeted	Medium	Ensuring primary research variables are complete.
axis=1	High	Low	Removing features with poor data collection quality.
inplace	High	Low	When memory efficiency is needed
ignore_index	High	High	Cleaning disorganized datasets where the index is purely positional.

Check your Progress 2

1. Create a pandas DataFrame with at least one missing value in a numeric column and one missing value in a text column. Use `df.isnull().sum()` to count the number of missing values in each column. Convert the tables (1.4.2.1, 1.4.2.2).

.....

.....

.....

2. Why is the default behavior of `dropna()` to drop rows rather than columns?

.....

.....

.....

3. What is the index of the default axis in pandas?

.....

.....

.....

4. If a Series object has missing values and the `ignore_index` parameter is set to True, what will be the resulting index?

.....

.....

.....

5. What does the parameter `thresh=4` do in `dropna(thresh=4)`?

.....

.....

.....

2.3.3 Filling in Missing Data

In our previous discussions, we explored how to identify and remove incomplete records. However, as a data scientist, you will often find that simply deleting data is too "expensive" in terms of information loss. In this unit, we will learn how to "fill in the gaps" using a process called **Imputation**. Let us try to understand how we can keep our datasets robust without throwing away valuable observations.

We have seen that deletion is easy, but it can destroy your model if you are not careful. A more refined and often more effective approach is imputation, which involves filling in missing values with a substituted value.

In the pandas library, our primary tool for this is the `fillna()` function. Let us prepare a sample dataset to see how this works in a real-world scenario, such as an HR employee database.

	Employee_ID	Department	Salary	Experience_Years	Join_Date
0	E101	HR	45000.0	2.0	2021-06-01
1	E102	Finance	54000.0	NaN	2020-08-15
2	E103	IT	NaN	5.0	NaN
3	E104	Finance	58000.0	7.0	2019-11-20
4	E105	NaN	60000.0	NaN	2021-02-10
5	E106	IT	NaN	3.0	2020-05-30
6	E107	HR	52000.0	4.0	NaN

```
import pandas as pd
import numpy as np
# Let us create a sample dataset with missing values
data = {
```



```
'Employee_ID': ['E101', 'E102', 'E103', 'E104', 'E105',  
'E106', 'E107'],  
'Department': ['HR', 'Finance', 'IT', 'Finance', np.nan,  
'IT', 'HR'],  
'Salary': [45000, 54000, np.nan, 58000, 60000, np.nan,  
52000],  
'Experience_Years': [2, np.nan, 5, 7, np.nan, 3, 4],  
'Join_Date': ['2021-06-01', '2020-08-15', np.nan, '2019-  
11-20', '2021-02-10', '2020-05-30', np.nan]  
}  
df = pd.DataFrame(data)  
print("Initial DataFrame with Missing Values:")  
print(df)  
# We can check the extent of missingness using isnull()  
print("\nCount of Missing Values per Column:")  
print(df.isnull().sum())
```

Output:

```
Initial DataFrame with Missing Values:  
  Employee_ID Department  Salary Experience_Years  Join_Date  
0      E101      HR    45000.0          2.0    2021-06-01  
1      E102  Finance    54000.0          NaN    2020-08-15  
2      E103      IT       NaN          5.0          NaN  
3      E104  Finance    58000.0          7.0    2019-11-20  
4      E105      NaN    60000.0          NaN    2021-02-10  
5      E106      IT       NaN          3.0    2020-05-30  
6      E107      HR    52000.0          4.0          NaN  
  
Count of Missing Values per Column:  
Employee_ID      0  
Department       1  
Salary           2  
Experience_Years  2  
Join_Date        2  
dtype: int64
```

Fill Missing Data with a Fixed Value

Sometimes, you might want to replace missing values with a specific "placeholder" or a default number. This is the simplest form of imputation.

How to do it:

We use `df.fillna(value)`. If we want to target a specific column, we call it on that column. You should remember that using `inplace=True` will modify your original DataFrame directly.

```
df_fixed = df.copy()  
# Replacing missing Departments with 'Unknown'  
df_fixed.fillna({'Department': 'Unknown'}, inplace=True)  
  
# Replacing missing Salaries with a default value of 50,000  
df_fixed.fillna({'Salary': 50000}, inplace=True)  
  
print("After Fixed Value Imputation:")  
print(df_fixed)
```

Output:

After Fixed Value Imputation:

	Employee_ID	Department	Salary	Experience_Years	Join_Date
0	E101	HR	45000.0	2.0	2021-06-01
1	E102	Finance	54000.0	NaN	2020-08-15
2	E103	IT	50000.0	5.0	NaN
3	E104	Finance	58000.0	7.0	2019-11-20
4	E105	Unknown	60000.0	NaN	2021-02-10
5	E106	IT	50000.0	3.0	2020-05-30
6	E107	HR	52000.0	4.0	NaN

Using Statistical Methods (Mean, Median, Mode)

Filling with a fixed value is simple, but it doesn't account for the nature of your data.

We can use statistics to be more precise:

Mean (Average): Best for "Normal" data that is evenly distributed.

Median (Middle Value): Best if your data has outliers (extreme values) that might skew the average.

Mode (Most Frequent): Best for categorical data (text or groups).

Let us try to apply these to our dataset:

```
df_stats = df.copy()
# Salary might be normally distributed, so we use mean
df_stats.fillna({'Salary':df_stats['Salary'].mean()},
inplace=True)
# Experience might have outliers, so median is safer
df_stats.fillna({'Experience_Years':df_stats['Experience_Years'].median()}, inplace=True)
# Department is text-based, so we use the most frequent one (Mode)
# Note:.mode() returns a Series, so we take the first element
df_stats.fillna({'Department':df_stats['Department'].mode()}
, inplace=True)
print("After Statistical Imputation:")
print(df_stats)
```

Output:

After Statistical Imputation:

Input DataFrame Input:					
	Employee_ID	Department	Salary	Experience_Years	Join_Date
0	E101	HR	45000.0	2.0	2021-06-01
1	E102	Finance	54000.0	4.0	2020-08-15
2	E103	IT	53800.0	5.0	NaN
3	E104	Finance	58000.0	7.0	2019-11-20
4	E105	NaN	60000.0	4.0	2021-02-10
5	E106	IT	53800.0	3.0	2020-05-30
6	E107	HR	52000.0	4.0	NaN

Forward Fill and Backward Fill

When dealing with time-series or ordered data, such as daily temperature readings or stock prices, missing values often resemble the values immediately preceding or following them. In such cases, we use "Forward Fill" (`ffill`) or "Backward Fill" (`bfill`).

Imagine a weather station recording humidity. If the 2:00 PM reading is missing, the 1:00 PM reading is likely a better estimate than the monthly average. However, you should be careful with "limit" parameters. Pandas allows you to set a limit so that you

don't fill gaps that are too large, which would compromise data integrity.

```
# Fill Using Forward Fill and Backward Fill
df_ffill = df.copy()
df_ffill['Join_Date'].fillna(method='ffill', inplace=True)
# forward fill
df_ffill['Experience_Years'].fillna(method='bfill',
inplace=True) # backward fill
print("\nFilling Missing Data using Forward/Backward Fill:")
print(df_ffill)
```

Output:

```
Filling Missing Data using Forward/Backward Fill:
Employee_ID Department Salary Experience_Years Join_Date
0 E101 HR 45000.0 2.0 2021-06-01
1 E102 Finance 54000.0 5.0 2020-08-15
2 E103 IT NaN 5.0 2020-08-15
3 E104 Finance 58000.0 7.0 2019-11-20
4 E105 NaN 60000.0 3.0 2021-02-10
5 E106 IT NaN 3.0 2020-05-30
6 E107 HR 52000.0 4.0 2020-05-30
```

Conditional Filling

This technique fills missing data based on conditions or rules. This is particularly valuable when domain knowledge provides insight into what missing values should be.

```
# Conditional Filling Example
df_cond = df.copy()
# Suppose we assume that IT employees' missing salary = 55000
df_cond.loc[(df_cond['Department'] == 'IT') &
(df_cond['Salary'].isnull()), 'Salary'] = 55000
# Suppose we assume HR employees with missing experience = 3
years
df_cond.loc[(df_cond['Department'] == 'HR') &
(df_cond['Experience_Years'].isnull()), 'Experience_Years'] = 3
print("\nConditional Filling Example:")
print(df_cond)
```

Output:

```
Conditional Filling Example:
Employee_ID Department Salary Experience_Years Join_Date
0 E101 HR 45000.0 2.0 2021-06-01
1 E102 Finance 54000.0 NaN 2020-08-15
2 E103 IT 55000.0 5.0 NaN
3 E104 Finance 58000.0 7.0 2019-11-20
4 E105 NaN 60000.0 NaN 2021-02-10
5 E106 IT 55000.0 3.0 2020-05-30
6 E107 HR 52000.0 4.0 NaN
```

While filling in missing values (what we call **imputation**) is usually a much better option than just throwing away our hard-earned data, you should know that it comes with its own set of unique challenges!

Let us try to visualize what happens when we use a simple approach, like filling in all our blank spaces with the average (the mean) of that column.

When you replace every single missing value with the exact same middle number, you may notice that your data suddenly looks a lot less diverse. By crowding the center, we

artificially shrink the natural spread of our data, which reduces both its overall variation and its standard deviation.

Because we are making so many rows look identical, we also weaken the real-world relationships, or correlations, between this column and the other variables in our dataset. This actually introduces a sneaky new form of bias into our analysis!

To get around this, you can eventually use more sophisticated methods, such as regression imputation (which uses other columns to predict the missing value) or multiple imputation. These are incredibly powerful tools, and while they are a bit advanced for our beginner's journey, it is wonderful to keep them on your radar as you continue to build your data science skills!

2.4 DATA TRANSFORMATION

By now, you have probably realized that raw data is rarely "ready-to-use." After we do basic cleaning (like handling blank spaces), we enter the phase of data transformation.

Think of data transformation as the process of polishing and reshaping our clean data so that it is in the absolute best format for analysis. It is the bridge between raw, chaotic data and high-performing machine learning models.

In this section, we will learn how to handle duplicate records, apply custom logic to our data, and rename parts of our datasets to make them more readable.

2.4.1 Finding and Removing Duplicate Records

Have you ever accidentally saved a contact twice in your phone? That's a duplicate.

Duplicate rows occur when the same information is recorded more than once. This often happens because of data entry errors, or when we combine multiple datasets. If we leave these redundancies in our data, they can distort our statistics, make our models inaccurate, and waste memory.

Basic Deduplication

To handle duplicates in Pandas, we use a wonderful method called `drop_duplicates()`! By default, this method looks at every single column in a row. If it finds a row that is identical to an earlier one, it marks it as a duplicate and discards it.

Behind the scenes, the Pandas engine uses a fast tracking mechanism (a hash table) to spot duplicates. This makes the method incredibly fast, running in a time complexity of approximately $O(n)$, where n is the number of rows.

But how does it handle missing values? In Pandas, null markers (like `NaN` or `None`) are treated as equal to other null markers. So, if two rows are identical except that they both have a missing value in the same column, Pandas will consider them duplicates and drop one of them.

```
import pandas as pd
import numpy as np
# (optional) show full columns when printing
pd.set_option('display.max_columns', None)
# Example dataset contains duplicates, inconsistent text,
missing values, and mixed date formats.
```

```
data = {
    'product_id':
    ['P101', 'P102', 'P103', 'P101', 'P104', 'P105', 'P106', 'P107', 'P108',
    , 'P105'],
    'product_name': ['Laptop', 'Toaster', 'Coffee
    Maker', 'Laptop', 'Smartphone', 'Blender', 'Headphones', 'TV', 'Micro
    wave', 'Blender'],
    'price': [150.0, np.nan, 250.50, 150.0, 400.0, np.nan,
    120.0, 800.0, 200.0, np.nan],
    'category':
    ['Electronics', 'HomeGoods', np.nan, 'Electronics', 'Electronics', '
    Appliances', ' Electronics
    ', 'electronics', 'HomeGoods', 'Appliances'],
    'rating': [4.5, 3.8, 4.0, np.nan, 4.2, 3.9, 4.1, 4.7, np.nan, 3.9],
    'launch_date': ['2024-05-01', '2023/11/12', '01-02-
    2022', '2024-05-01', '15 Jun 2024', '2022-08-30', None, '2024-07-
    20', '2023-01-15', '2022-08-30']
}
df = pd.DataFrame(data)
# quick look
print("Initial DataFrame (rows: {}, cols:
    {})".format(*df.shape))
print("Original DataFrame:")
print(df)
```

Output:

```
Initial DataFrame (rows: 10, cols: 6)
Original DataFrame:
  product_id product_name  price  category  rating launch_date
0      P101      Laptop   150.0  Electronics    4.5  2024-05-01
1      P102      Toaster    NaN   HomeGoods    3.8  2023/11/12
2      P103  Coffee Maker   250.5         NaN    4.0   01-02-2022
3      P101      Laptop   150.0  Electronics    NaN  2024-05-01
4      P104  Smartphone   400.0  Electronics    4.2  15 Jun 2024
5      P105      Blender    NaN   Appliances    3.9  2022-08-30
6      P106  Headphones   120.0  Electronics    4.1         None
7      P107         TV   800.0  electronics    4.7  2024-07-20
8      P108  Microwave   200.0   HomeGoods    NaN  2023-01-15
9      P105      Blender    NaN   Appliances    3.9  2022-08-30
```

Exploring subset and keep Configurations

Sometimes, defining a "duplicate" depends on your business goals. For instance, what if two rows represent the same product ID, but have different launch dates or ratings?

To handle this, Pandas gives us some handy arguments:

- **subset:** This tells Pandas to only look at specific columns to determine if a row is a duplicate.
- **keep:** This lets you choose which duplicate row to keep. You can choose 'first' (default), 'last', or False (which removes all duplicates completely!).
- **inplace:** If you set this to True, it alters our original DataFrame directly. If False (default), it leaves the original alone and gives us a nice, clean copy.

- `ignore_index`: When you drop rows, you might notice your index numbers look a bit jumpy (e.g., 0, 1, 3, 4). Setting this to `True` resets the row labels from `$0$` to `$n-1$` automatically.

Let us summarise these parameters in a simple, clear table:

Parameter	Type	Default	What it does
<code>subset</code>	List or String	<code>None</code>	Restricts identity evaluation to the specified columns.
<code>keep</code>	String or Bool	<code>'first'</code>	Choose what to keep: <code>'first'</code> , <code>'last'</code> , or <code>False</code> (drops all duplicates).
<code>inplace</code>	Boolean	<code>False</code>	Modifies the DataFrame directly if <code>True</code> , instead of returning a copy.
<code>ignore_index</code>	Boolean	<code>False</code>	Automatically resets the row numbers sequentially from 0 if <code>True</code> .

Let us see how these arguments behave under different configurations:

1. Default Behavior (`keep='first'`)

If we run `drop_duplicates()` without any arguments, it checks all columns and keeps the very first occurrence of each unique row.

```
df_cleaned_first = df.drop_duplicates()
print("DataFrame Keeping first (rows: {}, cols:
{}).format(*df_cleaned_first.shape))
print("DataFrame after dropping duplicates (keeping first):")
print(df_cleaned_first)
```

Output:

```
DataFrame Keeping first (rows: 9, cols: 6)
DataFrame after dropping duplicates (keeping first):
  product_id product_name price category rating launch_date
0      P101      Laptop  150.0  Electronics    4.5   2024-05-01
1      P102      Toaster   NaN    HomeGoods    3.8   2023/11/12
2      P103  Coffee Maker  250.5         NaN    4.0    01-02-2022
3      P101      Laptop  150.0  Electronics   NaN   2024-05-01
4      P104  Smartphone  400.0  Electronics    4.2  15 Jun 2024
5      P105      Blender   NaN    Appliances    3.9   2022-08-30
6      P106  Headphones  120.0  Electronics    4.1         None
7      P107         TV   800.0  electronics    4.7   2024-07-20
8      P108      Microwave  200.0    HomeGoods   NaN   2023-01-15
```

What happened here? Only Row 9 was dropped because it was an exact copy of Row 5. Row 3 remained because it has a missing rating (`NaN`), making it slightly different from Row 0 when evaluating all columns.

2. Keeping the Last Occurrence (`keep='last'`)

What if you want to keep the latest entry of a duplicate instead? Let us try `keep='last'`:

```
df_cleaned_last = df.drop_duplicates(keep='last')
print("Cleaned DataFrame Keeping last (rows: {}, cols:
{}).format(*df_cleaned_last.shape))
print("DataFrame after dropping duplicates (keeping last):")
print(df_cleaned_last)
```

Output:

```
Cleaned DataFrame Keeping last (rows: 9, cols: 6)
DataFrame after dropping duplicates (keeping last):
  product_id product_name price category rating launch_date
0      P101      Laptop  150.0  Electronics    4.5  2024-05-01
1      P102      Toaster   NaN    HomeGoods    3.8  2023/11/12
2      P103  Coffee Maker  250.5         NaN    4.0  01-02-2022
3      P101      Laptop  150.0  Electronics    NaN  2024-05-01
4      P104  Smartphone  400.0  Electronics    4.2  15 Jun 2024
6      P106  Headphones  120.0  Electronics    4.1         None
7      P107         TV   800.0  electronics    4.7  2024-07-20
8      P108  Microwave  200.0    HomeGoods    NaN  2023-01-15
9      P105      Blender   NaN    Appliances    3.9  2022-08-30
```

What happened here? Row 5 (the first Blender row) was dropped, and Row 9 (the last Blender row) was kept.

3. Wiping Out All Duplicates (keep=False)

Sometimes, any duplicate entry is suspicious, and we might want to delete them all. We can set `keep=False`:

```
df_cleaned_all = df.drop_duplicates(keep=False)
print("Wipping all duplicates (rows: {}, cols:
{}).format(*df_cleaned_all.shape))
print("DataFrame after dropping ALL duplicate occurrences:")
print(df_cleaned_all)
```

Output:

```
Wipping all duplicates (rows: 8, cols: 6)
DataFrame after dropping ALL duplicate occurrences:
  product_id product_name price category rating launch_date
0      P101      Laptop  150.0  Electronics    4.5  2024-05-01
1      P102      Toaster   NaN    HomeGoods    3.8  2023/11/12
2      P103  Coffee Maker  250.5         NaN    4.0  01-02-2022
3      P101      Laptop  150.0  Electronics    NaN  2024-05-01
4      P104  Smartphone  400.0  Electronics    4.2  15 Jun 2024
6      P106  Headphones  120.0  Electronics    4.1         None
7      P107         TV   800.0  electronics    4.7  2024-07-20
8      P108  Microwave  200.0    HomeGoods    NaN  2023-01-15
```

What happened here? Both Row 5 and Row 9 were deleted entirely, leaving only unique rows that appeared once in our raw data.

4. Dropping Duplicates Based on Specific Columns (subset)

What if you only care whether the `product_id` and `product_name` match? Let us try:

```
df_subset_cleaned = df.drop_duplicates(subset=["product_id",
"product_name"])
print("Cleaned Subset (rows: {}, cols:
{}).format(*df_subset_cleaned.shape))
print("DataFrame after dropping duplicates based on
'product_id' and 'product_name':")
print(df_subset_cleaned)
```

Output:

```
Cleaned Subset (rows: 8, cols: 6)
DataFrame after dropping duplicates based on 'product_id' and
'product_name':
  product_id product_name price category rating launch_date
```

0	P101	Laptop	150.0	Electronics	4.5	2024-05-01
1	P102	Toaster	NaN	HomeGoods	3.8	2023/11/12
2	P103	Coffee Maker	250.5	NaN	4.0	01-02-2022
4	P104	Smartphone	400.0	Electronics	4.2	15 Jun 2024
5	P105	Blender	NaN	Appliances	3.9	2022-08-30
6	P106	Headphones	120.0	Electronics	4.1	None
7	P107	TV	800.0	electronics	4.7	2024-07-20
8	P108	Microwave	200.0	HomeGoods	NaN	2023-01-15

What happened here? Row 3 is gone! Because we only checked `product_id` and `product_name`, the duplicate "Laptop" (Row 3) was identified and removed, even though its rating was missing.

Check Your Progress 3

1. Create a DataFrame that contains at least one row that is a complete duplicate of another row. Use the `drop_duplicates()` method to remove the duplicate row, keeping the first occurrence. Print the resulting DataFrame.

.....

2. Create a new DataFrame where two rows have the same values in a specific set of columns (`Name` and `City`) but different values in another column (`Age`). Use `drop_duplicates()` with the `subset` parameter to remove the duplicate based on `Name` and `City`, keeping the first occurrence.

.....

3. Explain the difference between `df.drop_duplicates(keep='first')` and `df.drop_duplicates(keep=False)`.

.....

2.4.2 Transforming Data: apply versus transform versus map

Once our dataset is deduplicated, we often need to transform the values. Pandas gives us three main options: `apply()`, `transform()`, and `map()`. While they look similar, they have very different rules! Let us understand their differences.

The Big Differences Between `apply()`, `transform()`, and `map()`

- `transform()` is like a strict layout controller. Whatever function you pass to it, the output must have the exact same length and index as your input. This makes it perfect for things like standardizing data or filling empty spaces with a group average. However, it cannot aggregate data (like calculating a single sum). If your function collapses the row count, `transform()` will throw a `ValueError`! It is also restricted to looking at one column at a time.
- `apply()` is your flexible helper. It can aggregate data (shrinking rows down to a single value) or keep it the same size. It also passes your entire DataFrame to the function, which means you can perform calculations that compare different columns at the same time!

- `map()` is a specialist. It only works on single columns (Series) and is used for fast, element-by-element substitutions, like using a dictionary to map codes to names. Let us summarise these in a table:

Feature	<code>apply()</code>	<code>transform()</code>	<code>map()</code>
Applicable on	DataFrames or Series	DataFrames or Series	Series only
Output Shape	Flexible: can expand, shrink, or match	Strict: must return the exact same size	Strict: always matches original size
Sees other columns?	Yes, can access multiple columns	No, restricted to one column	No, restricted to one column
Accepts	Callables or lambda functions	Callables, string names (like 'sqrt'), list of functions, or dicts	Callables, dictionaries, or Series

Translating Values with `apply()`

The Notebook Trap: The Idempotency Crisis!

Let us try applying a 10% price increase on any product priced over 500.

```
# Import necessary libraries
import pandas as pd
import numpy as np
# Let's make a new dataset
data = {
    'product_id': ['P101', 'P102', 'P103', 'P104', 'P105'],
    'category': ['Electronics', 'Furniture', 'Electronics',
                'Clothing', 'Clothing'],
    'price': [1200, 850, 300, 450, 500],
    'discount_code': ['D10', 'D15', 'D05', np.nan, 'D20']
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
```

Output:

```
Original DataFrame:
  product_id  category  price  discount_code
0      P101  Electronics   1200           D10
1      P102   Furniture    850           D15
2      P103  Electronics    300           D05
3      P104   Clothing    450           NaN
4      P105   Clothing    500           D20
```

```
# Define our transformation function
def increase_price(x):
    return x * 1.10 if x > 500.0 else x
# Apply the transformation
df['price'] = df['price'].apply(increase_price)
print("\nAfter price adjustment:")
print(df)
```

Output:

After price adjustment:				
	product_id	category	price	discount_code
0	P101	Electronics	1320.0	D10
1	P102	Furniture	935.0	D15
2	P103	Electronics	300.0	D05
3	P104	Clothing	450.0	NaN
4	P105	Clothing	500.0	D20

Why is this a danger in Jupyter Notebooks?

Let us discuss a major trap! What happens if you run the Jupyter cell containing `df['price'] = df['price'].apply(increase_price)` multiple times?

Because we overwrite `df['price']`, the new price of 1320 is still greater than 500. So, running it a second time will multiply 1320 by another 10%! Running it three times will compound the increase.

Mathematically, this follows the compound formula:

$$P_t = P_0 \times (1 + r)^t$$

If you run it three times, the price of 1200 becomes:

$$1200 \times (1.10)^3 = 1597.20$$

However, the price of 500.0 is not strictly greater than 500, so it remains unchanged no matter how many times you run the cell! This mismatch is a huge data integrity risk.

To keep your code **idempotent** (meaning you get the same result no matter how many times you run it), let us try creating a new column instead of overwriting our original:

```
df['adjusted_price'] = df['price'].apply(increase_price)
print("Increasing Price by 10%:")
print(df)
```

Output:

Increasing Price by 10%:					
	product_id	category	price	discount_code	adjusted_price
0	P101	Electronics	1320.0	D10	1452.0
1	P102	Furniture	935.0	D15	1028.5
2	P103	Electronics	300.0	D05	300.0
3	P104	Clothing	450.0	NaN	450.0
4	P105	Clothing	500.0	D20	500.0

What happened here? A new column `adjusted_price` has been added.

Translating Values with `map()`

If you want to map categories or codes, `map()` with a dictionary is extremely easy and clean. Let us try converting codes to percentage figures:

```
# Map discount codes to numeric percentages
discount_mapping = {'D05': 5.0, 'D10': 10.0, 'D15': 15.0,
                    'D20': 20.0}

df['discount_percent'] =
df['discount_code'].map(discount_mapping)
print("Adding a column Discount Percent using a mapping
dictionary:")
print(df)
```

Output:

```
Adding a column Discount Percent using a mapping dictionary:
  product_id  category  price  discount_code  adjusted_price  \
0      P101  Electronics  1320.0           D10          1452.0
1      P102   Furniture   935.0           D15          1028.5
2      P103  Electronics   300.0           D05           300.0
3      P104   Clothing   450.0           NaN           450.0
4      P105   Clothing   500.0           D20           500.0

  discount_percent
0              10.0
1              15.0
2              5.0
3              NaN
4             20.0
```

Notice how NaN values are preserved perfectly!

2.4.3 Replacing Values and Renaming Columns or Row Indexes

Replacing Specific Values

If you just want to swap out a few values, the `replace()` method is incredibly clear. Replacing specific values is a common task in data cleaning, especially when dealing with inconsistent data entry or placeholders. The `replace()` method in pandas is a flexible tool for this purpose. It can be used to replace single values, lists of values, or even patterns using regular expressions.

```
# Replace 'Clothing' with 'Apparel' in the category column
df['category'] = df['category'].replace('Clothing', 'Apparel')
# Replace multiple values at once using a dictionary
df.replace({'category': {'Furniture': 'Home Decor',
                        'Electronics': 'Gadgets'}}), inplace=True)
print("Replacing some values in category:")
print(df)
```

Output:

```
Replacing some values in category:
  product_id  category  price  discount_code  adjusted_price  \
0      P101   Gadgets  1320.0           D10          1452.0
1      P102 Home Decor   935.0           D15          1028.5
2      P103   Gadgets   300.0           D05           300.0
3      P104   Apparel   450.0           NaN           450.0
4      P105   Apparel   500.0           D20           500.0

  discount_percent
0              10.0
1              15.0
2              5.0
3              NaN
4             20.0
```

Renaming Column and Row Headers

To keep your code easy to read, let us make sure our columns and indexes are cleanly named. We can do this in two main ways:

1. Renaming selectively with `rename()`

Using `rename()` with a dictionary is the safest approach because you only change the specific columns you want, leaving the rest untouched.

```
# Renaming columns using a dictionary with .rename()
new_names_dict = {'product_id': 'ProductID', 'category':
'Category'}
df_renamed = df.rename(columns=new_names_dict)
print("DataFrame after renaming specific columns:")
print(df_renamed)
```

Output:

```
DataFrame after renaming specific columns:
  ProductID  Category  price discount_code  adjusted_price \
0      P101    Gadgets  1320.0          D10          1452.0
1      P102  Home Decor   935.0          D15          1028.5
2      P103    Gadgets   300.0          D05           300.0
3      P104    Apparel   450.0          NaN           450.0
4      P105    Apparel   500.0          D20           500.0

  discount_percent
0              10.0
1              15.0
2               5.0
3              NaN
4              20.0
```

2. Renaming everything with Direct Assignment

You can assign a list of names directly to `df.columns`. But watch out! The list must have the exact same length as your column count, or Pandas will throw a `ValueError`

```
# Renaming a column using direct assignment (renames all
columns)
df.columns = ['ProductID', 'Category', 'Price', 'Discount
Code', 'Adjusted Price', 'Discount Percent']
print("DataFrame after renaming all columns using direct
assignment:")
print(df)
```

Output:

```
DataFrame after renaming all columns using direct assignment:
  ProductID  Category  Price Discount Code  Adjusted Price \
0      P101    Gadgets  1320.0          D10          1452.0
1      P102  Home Decor   935.0          D15          1028.5
2      P103    Gadgets   300.0          D05           300.0
3      P104    Apparel   450.0          NaN           450.0
4      P105    Apparel   500.0          D20           500.0

  Discount Percent
0              10.0
1              15.0
2               5.0
3              NaN
4              20.0
```

3. Renaming Row Indexes

We can also rename our row indexes directly or selectively:

```
# Renaming row indexes directly
df.index = ['A', 'B', 'C', 'D', 'E']
print("DataFrame after renaming row indexes:")
print(df)
```

Output:

```
DataFrame after renaming row indexes:
  ProductID  Category  Price Discount  Code  Adjusted Price \
A      P101   Gadgets  1320.0          D10      1452.0
B      P102  Home Decor   935.0          D15      1028.5
C      P103   Gadgets   300.0          D05       300.0
D      P104   Apparel   450.0          NaN      450.0
E      P105   Apparel   500.0          D20      500.0

Discount Percent
A          10.0
B          15.0
C           5.0
D           NaN
E          20.0
```

What happened here? The row indices have now changed from numbers to letters.

4. Renaming rows selectively with rename()

You can also selectively rename different rows, as shown by the Python code below:

```
# Rename specific indexes with a dictionary
new_index_dict = {'A': 'first_row', 'E': 'last_row'}
df.rename(index=new_index_dict, inplace=True)
print("DataFrame after renaming specific row indexes:")
print(df)
```

Output:

```
DataFrame after renaming specific row indexes:
  ProductID  Category  Price Discount  Code  Adjusted Price \
first_row   P101   Gadgets  1320.0          D10      1452.0
B          P102  Home Decor   935.0          D15      1028.5
C          P103   Gadgets   300.0          D05       300.0
D          P104   Apparel   450.0          NaN      450.0
last_row    P105   Apparel   500.0          D20      500.0

Discount Percent
first_row      10.0
B              15.0
C               5.0
D               NaN
last_row       20.0
```

5. Lambda Sanitization, add_prefix, and add_suffix

If you have columns with messy spaces or capitalization, you can sanitize them using a lambda function with rename():

```
# Let's remove spaces and convert everything to lowercase
df.rename(columns=lambda x: x.strip().replace(' ',
'_').lower()), inplace=True)
print("DataFrame after renaming using lambda:")
print(df)
```

Output:

```
DataFrame after renaming using lamdda:
  productid  category  price discount_code  adjusted_price
\
first_row   P101   Gadgets  1320.0          D10      1452.0
```

B	P102	Home Decor	935.0	D15	1028.5
C	P103	Gadgets	300.0	D05	300.0
D	P104	Apparel	450.0	NaN	450.0
last_row	P105	Apparel	500.0	D20	500.0

	discount_percent	
first_row		10.0
B		15.0
C		5.0
D		NaN
last_row		20.0

You can also prepend or append tags to all your column headers using `add_prefix()` or `add_suffix()`. However, for this purpose you need to create a new dataframe:

```
df1=df.add_prefix("col_") # Becomes col_ProductID, col_Category
print("DataFrame after adding prefix in columns:")
print(df1)
df2=df.add_suffix("_col") # Becomes ProductID_col, Category_col
print("DataFrame after adding suffix in columns:")
print(df2)
```

Output:

```
DataFrame after adding prefix in columns:
   col_productid col_category col_price col_discount_code \
first_row      P101      Gadgets    1320.0             D10
B              P102  Home Decor     935.0             D15
C              P103      Gadgets     300.0             D05
D              P104      Apparel     450.0             NaN
last_row       P105      Apparel     500.0             D20

   col_adjusted_price  col_discount_percent
first_row          1452.0                10.0
B              1028.5                15.0
C              300.0                 5.0
D              450.0                 NaN
last_row         500.0                20.0
DataFrame after adding suffix in columns:
   productid_col category_col price_col discount_code_col \
first_row      P101      Gadgets    1320.0             D10
B              P102  Home Decor     935.0             D15
C              P103      Gadgets     300.0             D05
D              P104      Apparel     450.0             NaN
last_row       P105      Apparel     500.0             D20

   adjusted_price_col  discount_percent_col
first_row          1452.0                10.0
B              1028.5                15.0
C              300.0                 5.0
D              450.0                 NaN
last_row         500.0                20.0
```

Check your progress 4

1. Create a new column called `final_price` by subtracting the discount percent from the price in our sample product DataFrame.

Hint: The formula is:

$$P_{final} = P \times \left(1 - \frac{D}{100}\right)$$

.....

.....

.....

2. Write a Python function called `price_category` that takes a price value as input and returns "Budget" if the price is less than 500, "Standard" if it's between 500 and 1000, and "Premium" if it's greater than 1000. Apply this to our price column using `apply()` and save it as `price_tier`.

.....
.....
.....

3. Use the `rename()` method with a dictionary to change the column name `price_tier` to `product_price_tier`.

.....
.....
.....

4. If you want to apply a complex, multi-line function to a column in a Pandas DataFrame, which method is the most appropriate: `rename()`, `replace()`, or `apply()`? Why?

.....
.....
.....

5. You are combining sales data from two regions. You find that some orders have the same `Customer_ID` and `Order_Date` but slightly different `Shipping_Cost` due to system errors. Write the code to find and remove these duplicate orders based only on `Customer_ID` and `Order_Date`, keeping the first entry.

.....
.....
.....

2.5 SUMMARY

In this unit, we covered the fundamental and indispensable steps of Data Cleaning and Preparation, the backbone of any robust data science project.

We started by clarifying that Data Cleaning detects and fixes errors (missing values, typos, duplicates), while Data Transformation structures the cleaned data for analysis (creating new features, renaming).

We then focused on **Handling Missing Data**, distinguishing between **MCAR**, **MAR**, and **MNAR** data types. We learned to address these gaps using two primary methods:

- **Deletion** (`dropna()`) to remove incomplete rows or columns, noting the risk of information loss.
- **Imputation** (`fillna()`) to substitute missing values with statistical measures (mean, median) or constant values.

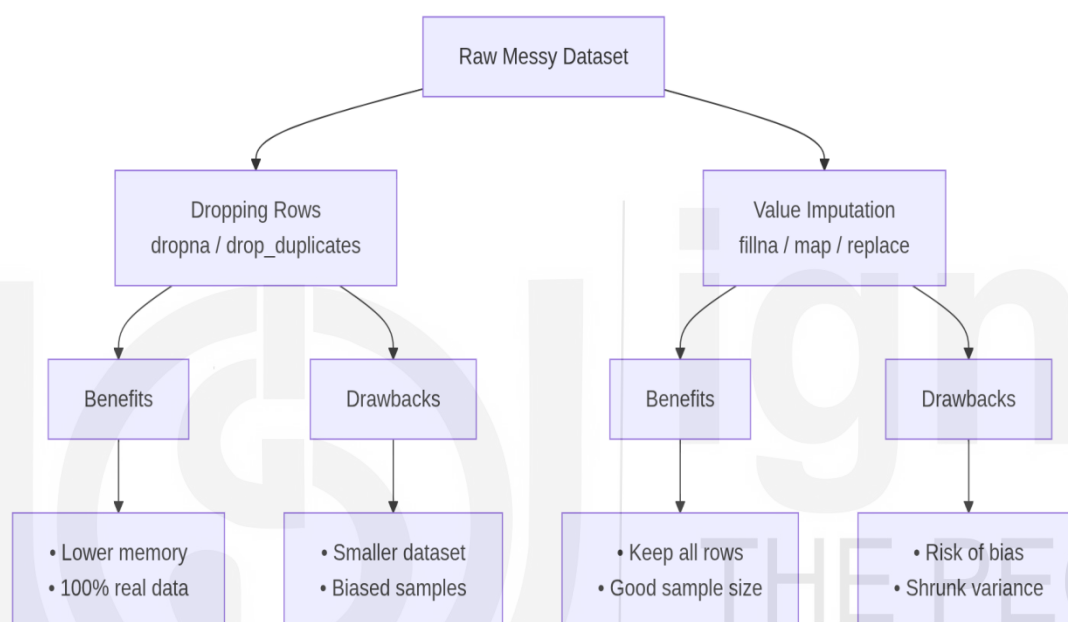
Finally, we explored Data Transformation techniques crucial for feature preparation:

- **Removing Duplicates** using `drop_duplicates()`, specifying subset criteria and control over which occurrence to keep.

- **Applying Functions** (`apply()`) for complex, element-wise transformations and creating derived features.
- **Replacing Values** (`replace()`) for standardizing inconsistent string or numeric data.
- **Renaming Axis Indexes** (`rename()`) to improve the clarity of our final dataset structure.

Mastering these pandas techniques ensures that our data is accurate, reliable, and ready to deliver meaningful insights to any analytical model.

When clean datasets are prepared, we always balance two pathways:



If we choose to drop duplicates or null rows, we get clean data, but we risk losing rows. If we choose to impute values, we preserve our dataset size, but we must be careful not to introduce synthetic bias.

As a data analyst, understanding these trade-offs is what makes you an exceptional data scientist. Keep practicing, run these notebooks, and enjoy your journey!

2.6 SOLUTIONS/ANSWERS

Check Your Progress 1

1. Because real-world data is inherently dirty and frequently contains missing values, duplicates, formatting errors, and outliers that necessitate intensive manual inspection and preprocessing, data cleaning takes a long time. It is regarded as essential since the dependability of the input data immediately affects the quality of any downstream analysis, machine learning model, or business insight. In the end, hurrying or omitting this phase ensures faulty results, which is entirely consistent with data science's "garbage in, garbage out" tenet.
2. This is a classic issue that almost every data professional runs into when working with real-world datasets!

The primary data cleaning activity that addresses this problem is **Data Standardisation** (which you will also see referred to as **Fixing Structural Errors** or **Categorical Text Normalisation**).

3. False

In the data world, we always perform Data Cleaning first. This is where we detect errors, remove pesky duplicate rows, and handle any missing data. Once we have a sparkling clean dataset, we move on to Data Transformation. This is where we reshape, scale, or map our clean data into the absolute best format for our final analysis or machine learning models. So, cleaning always sets the stage for transformation!

4. This is a wonderful example of Data Transformation (specifically, creating a derived variable, which is also commonly called **feature engineering**).

Check Your Progress – 2

1.

```
import pandas as pd
import numpy as np
# Define the data with missing values
data = {
    'Name': ['Alice', 'Bob', np.nan, 'Diana', 'Evan'], # Text
    'Age': [25.0, 30.5, 28.0, np.nan, 22.0], #
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR'] #
}
# Create the DataFrame
df = pd.DataFrame(data)
# Display the DataFrame
print("Initial DataFrame:")
print(df)
# Count missing values in each column
missing_values_count = df.isnull().sum()
# Display the result
print("\nMissing Values Count:")
print(missing_values_count)
```

Output:

```
Initial DataFrame:
   Name  Age Department
0  Alice  25.0         HR
1   Bob  30.5         IT
2   NaN  28.0    Finance
3  Diana   NaN         IT
4   Evan  22.0         HR

Missing Values Count:
Name      1
Age       1
Department 0
dtype: int64
```


2. In standard data analysis, rows represent unique observations. It is more common to discard an incomplete observation than to discard an entire variable that might be useful for other complete observations.
3. The default axis is 0, which corresponds to the 'index' or rows.
4. The resulting Series will have its index relabeled sequentially starting from 0.
5. thresh=4 keeps only those rows that contain at least 4 non-NaN values. Rows with fewer than 4 non-missing values are dropped.

Check Your Progress 3

1.

```
import pandas as pd
# Let's create a DataFrame with an exact duplicate (Row 0 and Row 2)
data_dup = {
    'employee_id': ['E101', 'E102', 'E101', 'E103'],
    'department': ['Engineering', 'Marketing', 'Engineering', 'Finance']
}
df_dup = pd.DataFrame(data_dup)
print("Original:")
print(df_dup)
# Drop exact duplicates and keep the first occurrence
df_dup_clean = df_dup.drop_duplicates(keep='first')
print("\nCleaned:")
print(df_dup_clean)
```

Output:

```
Original:
  employee_id  department
0         E101  Engineering
1         E102   Marketing
2         E101  Engineering
3         E103    Finance

Cleaned:
  employee_id  department
0         E101  Engineering
1         E102   Marketing
3         E103    Finance
```

2.

```
# Create a DataFrame where Alice has two different ages but the same City
data_sub = {
    'Name': ["Alice", "Bob", "Alice", "Charlie"],
    'City': ["Seattle", "Boston", "Seattle", "Denver"],
    'Age': [25, 30, 26, 35]
}
df_sub = pd.DataFrame(data_sub)
print("Original:")
print(df_sub)
# Deduplicate based on Name and City only, keeping the first entry
```

```
df_sub_clean = df_sub.drop_duplicates(subset=['Name',  
'City'], keep='first')  
print("\nCleaned with Subset:")  
print(df_sub_clean)
```

Output:

```
Original:  
   Name    City  Age  
0  Alice Seattle   25  
1   Bob   Boston   30  
2  Alice Seattle   26  
3 Charlie  Denver   35  
  
Cleaned with Subset:  
   Name    City  Age  
0  Alice Seattle   25  
1   Bob   Boston   30  
3 Charlie  Denver   35
```

3.

- keep='first': Keeps the first duplicate row it finds and discards any copies that appear after.
- keep=False: Completely deletes all instances of duplicate rows. If a row appears more than once, every single copy is wiped out of the dataset.

Check your progress - 4

1.

```
# Import necessary libraries  
import pandas as pd  
import numpy as np  
# Let's make a new dataset  
data = {  
    'product_id': ['P101', 'P102', 'P103', 'P104', 'P105'],  
    'category': ['Electronics', 'Furniture', 'Electronics',  
'Clothing', 'Clothing'],  
    'price': [1200, 850, 300, 450, 500],  
    'discount_code': ['D10', 'D15', 'D05', 'D00', 'D20']  
}  
df = pd.DataFrame(data)  
print("Original DataFrame:")  
print(df)  
# Map discount codes to numeric percentages  
discount_mapping = {'D00': 0, 'D05': 5.0, 'D10': 10.0, 'D15':  
15.0, 'D20': 20.0}  
df['discount_percent'] =  
df['discount_code'].map(discount_mapping)  
print("\nAdding a column Discount Percent using a mapping  
dictionary:")  
print(df)  
# Calculate final price. We use.fillna(0) to treat missing dis-  
count percents as 0%!  
df['final_price'] = df['price'] * (1.0 -  
df['discount_percent'].fillna(0.0) / 100.0)  
print("\nAdding a column Final Price:")  
print(df[['price', 'discount_percent', 'final_price']])
```

Output:

```
Original DataFrame:  
   product_id  category  price  discount_code  
0         P101  Electronics   1200          D10  
1         P102   Furniture    850          D15  
2         P103  Electronics    300          D05
```

3	P104	Clothing	450	D00
4	P105	Clothing	500	D20

Adding a column Discount Percent using a mapping dictionary:

	product_id	category	price	discount_code	discount_percent
0	P101	Electronics	1200	D10	10.0
1	P102	Furniture	850	D15	15.0
2	P103	Electronics	300	D05	5.0
3	P104	Clothing	450	D00	0.0
4	P105	Clothing	500	D20	20.0

Adding a column Final Price:

	price	discount_percent	final_price
0	1200	10.0	1080.0
1	850	15.0	722.5
2	300	5.0	285.0
3	450	0.0	450.0
4	500	20.0	400.0

2.

```
# Import necessary libraries
import pandas as pd
import numpy as np
# Let's make a new dataset
data = {
    'product_id': ['P101', 'P102', 'P103', 'P104', 'P105'],
    'category': ['Electronics', 'Furniture', 'Electronics',
                'Clothing', 'Clothing'],
    'price': [1200, 850, 300, 450, 500],
    'discount_code': ['D10', 'D15', 'D05', 'D00', 'D20']
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
# Define our classification function
def price_category(price_val):
    if price_val < 500.0:
        return "Budget"
    elif 500.0 <= price_val <= 1000.0:
        return "Standard"
    else:
        return "Premium"
# Apply our function using apply()
df['price_tier'] = df['price'].apply(price_category)
print("\nAdding a column Price Tier:")
print(df[['price', 'price_tier']])
```

Output:

Original DataFrame:

	product_id	category	price	discount_code
0	P101	Electronics	1200	D10
1	P102	Furniture	850	D15
2	P103	Electronics	300	D05
3	P104	Clothing	450	D00
4	P105	Clothing	500	D20

Adding a column Price Tier:

	price	price_tier
0	1200	Premium
1	850	Standard
2	300	Budget
3	450	Budget
4	500	Standard

3.

```
# Rename column 'price_tier' to 'product_price_tier'
df.rename(columns={'price_tier': 'product_price_tier'},
          inplace=True)
print("Updated columns:", df.columns.tolist())
```

Output:

```
Updated columns: ['product_id', 'category', 'price', 'discount_code',  
'product_price_tier']
```

4.

The best method is apply()!

- rename() is only used for changing label names of columns or rows.
- replace() is designed for swapping specific values directly, not for running custom multi-line functions.
- apply() lets us pass any custom Python function to process our data row-by-row or column-by-column.

5.

```
import pandas as pd  
# Create combined sales data  
sales_data = {  
    'Customer_ID': ['C400', 'C502', 'C400', 'C609'],  
    'Order_Date': ['2026-05-10', '2026-05-11', '2026-05-10',  
    '2026-05-12'],  
    'Shipping_Cost': [15.20, 10.50, 14.80, 22.00]  
}  
df_sales = pd.DataFrame(sales_data)  
  
# Remove duplicates based only on Customer_ID and Order_Date  
df_sales.drop_duplicates(subset=['Customer_ID', 'Order_Date'],  
keep='first', inplace=True)  
print(df_sales)
```

Output:

	Customer_ID	Order_Date	Shipping_Cost
0	C400	2026-05-10	15.2
1	C502	2026-05-11	10.5
3	C609	2026-05-12	22.0

2.7 REFERENCES AND FURTHER READINGS

1. McKinney, W. (2022). Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter. 3rd Edition. O'Reilly Media.
2. Bruce, P., Bruce, A., & Gedeck, P. (2020). Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python. O'Reilly Media.
3. Pandas Documentation: "Working with missing data."
https://pandas.pydata.org/docs/user_guide/missing_data.html
4. Medium Article: "From Messy to Magic: A Beginner-to-Expert Guide on Data Cleaning" by Nayeem Islam.
5. GeeksforGeeks: "Handling Missing Values in Python."
<https://www.geeksforgeeks.org/working-with-missing-data-in-pandas/>
6. Tableau Guide: "Data Cleaning: Definition, Benefits, and How-To."